Handoff 27: Python unit 3, part 1 (EDU-03: while loops, stopping on time, for and range)

Date: 2026-10-28 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#PR), not merged, not deployed. **No database migration, no new dependency, no API change.** Units 1 and 2 are untouched. One bug fix in the Python worker (section 5).

> **How to use this file:** upload it with `build.md`, `docs/plan/teaching-design.md`, `docs/plan/lesson-feedback.md`, `docs/plan/browser-checks.md` and handoffs 24 and 26. The pull request diff is the source of truth (`build.md` section 14). The appendix has every card, hint and check case word for word.

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 27 (EDU-03 part 1): the first
three lessons of Python unit 3 "Repeat and explore" (py.while-loops,
py.loop-termination, py.for-range), released with more lessons still to
come. I'm attaching build.md, teaching-design.md, lesson-feedback.md,
browser-checks.md, handoffs 24 and 26 and this handoff. Read the
appendix: check that the wording is calm, original and clear for a
grade 9 reader; that each check's cases catch loops that stop one step
early or late, never stop, or read too many replies, while accepting
other correct programs; that no hint, tagged hint or similar example
gives the answer; and that the cards are accurate Python. Explain at a
first-year computer engineering level.
```

1. Step 0 (checks before building)

Check	Result
`git fetch --all`, `gh pr list --state all`	Done. No open PRs
PR #31 (EDU-04 part 2) merged into `main`?	**Yes**, 2026-10-28 00:25 UTC (`4b70a28`)

Check	Result
Unit 3 work on any branch?	**None**: no `py/u03` lesson file on any local or remote branch
Where the work happened	A separate worktree, `../python-academy-u03`, on a new branch `content/py-u03-part1` from `origin/main` at `4b70a28`. The main folder was not switched. Not stacked on any PR

2. Handoff record (build.md template)

Field	Value
Task	EDU-03 part 1: `py.while-loops`, `py.loop-termination`, `py.for-range`; release `py.u03` with `more_lessons_coming: true`
Human goal	A learner can start unit 3 live, write loops that always finish, and is never told the unit is complete or shown what comes after it
Repository and branch	`torontojohn/python-academy`, branch `content/py-u03-part1`
Commits	`2012d11`: lessons, the worker fix, a new error translation, tests. `0dc0f0a`: course-wide counts and a re-recorded UI-05 fixture. Then docs and this handoff
Implementer / reviewer	Claude Code / the owner and a chat assistant
Non-goals	The rest of unit 3 (running totals, nested loops); a unit project or unit check; construct rule IDs; any change to units 1 or 2; `backlog.md` and `decision-log.md`
Data and compatibility	No migration. New lesson, task and badge IDs only. No API shape change: unit 3 uses the `more_lessons_coming` field and `complete` API field that PR #31 added
Deployment status	Not requested

Files changed

File	Why
`backend/content/py/u03/py.while-loops.json`, `py.loop-termination.json`, `py.for-range.json` (new)	The three lessons (appendix)
`backend/content/catalog.json`	`py.u03` released with three lessons, `more_lessons_coming: true`, and a summary that is true now: "while and for loops that always finish. Running totals and loops inside loops come next."

File	Why
`frontend/public/python-worker.js`	**Bug fix**: output a case left in Python's buffer no longer appears in the next case's output (section 5)
`frontend/app/feedback.ts`	A new translation, `indent-loop`, for a loop body that is not indented; the endless-loop (timeout) explanation now says what to check
`frontend/tests/fixtures/translations.snapshot.json`, `frontend/tests/feedback.spec.ts`, `frontend/tests/fixtures/pyodide-errors.json`, `docs/plan/lesson-feedback.md`	Kept in step with those two translation changes
`backend/tests/test_unit3_lessons.py` (new, 90 tests), `frontend/tests/unit3.spec.ts` (new, 29 tests), `frontend/tests/fixtures/unit3-api.json` (new, recorded real answers)	Section 6
`backend/tests/case_runner.py`	A 0.5-second limit per case, so a test sample with an endless loop is reported as `timeout` instead of hanging the suite
`backend/tests/test_unit2_lessons.py`	The hint audit can now assemble a loop with more than one line inside it (section 5); unit 2's results are unchanged
`frontend/tests/browser-checks.spec.ts` (1 new test)	Leftover output never leaks into the next case
`backend/tests/test_recommend.py` (3 new tests), `test_unit2_part2.py`, `test_content.py`, `test_lesson_units.py`, `test_academy.py`, `frontend/tests/academy.spec.ts`, `unit2.spec.ts`; fixtures `catalog_v1.json`, `lesson-units-api.json`, `unit2-api.json`, `unit2-part2-api.json`	14 lessons in the course, "3 of 12 units open", unit 3 released, unit 4 now the first coming-soon unit
`backend/content/README.md`, `docs/plan/browser-checks.md`	Unit 3 is the first unit released in parts; the "state between cases" row
`docs/handoffs/2026-10-28-27-unit3-part1.md` and `.pdf`	This handoff

3. How it works

The story continues from unit 2: the riddle keeper's secret number. Lesson 1 lets the player keep guessing (`while`); lesson 2 gives them only three tries (a loop that stops on time); lesson 3 walks steps with `for` and `range()`. Each lesson has the five cards of units 1 and 2, predict-then-run on cards 3 and 4, and fading support: lesson 1's starter has the two input lines and a comment, lesson 2's only comments, lesson 3's one comment.

Lesson	Challenge	The traps the cases catch
`py.while-loops` "Keep guessing"	While the guess is wrong, print `True` (too high) or `False` (too low), then ask again	A loop that never asks again (endless); one that asks once too often (runs out of replies); `if` instead of `while`; a right first guess (prints nothing)
`py.loop-termination` "Three tries"	At most three guesses: print how many were used, then whether the last was right	No limit (a fourth guess is read); `<=` for `<` (four tries); stopping one early; counting wrong; extra replies left unread
`py.for-range` "Step by step"	Print 2, 4, … up to the number given	`range(2, top)` missing the top when it is even; starting at 0; a top of 1 (prints nothing)

The four loop investigations you asked for

1. **Input running out.** A loop that asks for one reply too many raises `EOFError` in Python. - **In a check:** the case is `needs_work` with reason `missing_input`: "Your program asked for more replies than this check gives. Check how many times it calls input()." The lesson's tagged `EOFError` hint is offered first, and says what to check in this loop (lesson 1: "Check that it stops as soon as the guess is right"). - **On Try it:** the page shows the existing "Not enough player replies" message; card 4 of lesson 1 invites this on purpose.
2. **Endless loops.** - **In a check:** each case has 3 seconds; the program is stopped, the rest of the cases are not run, and the explanation now reads "…usually because a loop never ends. Check that something inside the loop changes, so that the loop can stop." - **On Try it:** a silent endless loop is stopped by the worker's existing 15-second limit, and the page stays usable. A printing one hits the 20,000-character Try it cap almost at once. Lesson 2's Try it (delete `count = count - 1`) is a printing loop, so the learner sees "Too much output" quickly instead of waiting.
3. **The output limit.** 10,000 characters per case in a check (unchanged). A printing endless loop reaches it in well under a second and is reported as `output_limit`. Every expected output in these lessons is under 200 characters (tested). Investigating this found the worker bug in section 5.
4. **Termination off-by-one.** Lesson 2 has two visible cases, `extra-replies` (more replies than tries; checked before `out-of-tries`) and `out-of-tries` (exactly three wrong). Order matters: - A loop with **no limit** reads a fourth reply in `extra-replies`, so it prints `4` and `True` instead of `3` and `False`. That is a wrong-output result with a tagged hint about the limit, not a confusing "ran out of replies". - A loop that **stops early** (`< 2`) fails `third-try`. - A loop that **stops late** (`<= 3`) fails `extra-replies`. - Lesson 3's `even-top` and `none` cases catch `range()` stopping one step early or running when it should not.

Constructs are not forced. There are no `require` rules. A `for` loop cannot be forced by behaviour (a `while` loop prints the same lines), so lesson 3's "Other ways" shows the `while` version and says why `for` is shorter. Lesson 1 could be written with `while True:` and `break`; that passes too and is in "Other ways".

Unit state.

Where	What the learner sees
Catalog	Unit 3 "Released", three lessons, the summary above; "3 of 12 units open"
Lesson page, after lesson 3	"You finished every lesson released so far…" (never "Unit complete")
Home recommendation, after the three lessons	"Python Foundations unit 3, Repeat and explore, has more lessons on the way. You have finished every lesson in it so far." Unit 4 is never named
Home recommendation, after unit 2	Continue with unit 3, lesson 1

`RULES_VERSION` stays **2**: the recommendation rules did not change. PR #31 already made a unit with more lessons coming "not done"; unit 3 is the first real unit to use it.

4. Decisions and why

Decision	Options considered	Reason	Who decided
Continue the riddle-keeper story	A new story for unit 3	The learner already knows the guessing game, so the new idea (repeating) is the only new thing	Implementer (**assumption 1**)
No construct rules; `for` shown as another way	A `uses-for` or `uses-while` rule	Your rule; and a `for` loop cannot be told apart from a `while` loop by what it prints	Owner rule
`RULES_VERSION` unchanged (2)	Bump to 3	Your rule: bump only if the rules change. They did not; only the catalog data did	Owner rule
`extra-replies` visible and before `out-of-tries`	Only `out-of-tries`	Without it, a missing limit shows "ran out of replies", which is true but does not point at the limit	Implementer
Fix the worker's leftover-output bug in this PR	A separate PR	Found while testing this unit's output-limit case; three lines; without it, a loop lesson's feedback could show another case's output	Implementer (**assumption 2**: you may want a backlog row for it)

Decision	Options considered	Reason	Who decided
New `indent-loop` translation	Keep the generic "unexpected indent" text	Python's message for a loop with no indented body is the first error many learners see in this unit; the generic text says "if"	Implementer
The timeout explanation names loops, **for every unit**	A unit 3-only text	The explanation is shared by all checks, and an endless loop is the usual cause in every unit (**assumption 3**)	Implementer
The test runner stops a case after 0.5 s	The browser's 3 s	Test samples only; 0.5 s is ample for these programs and keeps the suite fast	Implementer
One practice badge per lesson	None per lesson	As in units 1 and 2 (handoff 24 question 6 still open)	Owner

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
Worker leak: output from a case stopped at the output limit appeared at the start of the next case's output. Python's buffer still held text when the worker stopped the program, and it was printed during the next case	The recorded request for "prints for ever" did not match: the second case's reported output started with the first case's text	The worker flushes and discards Python's buffer before each case (`python-worker.js`). A new browser test fails without it (break B4)
An older recording had the leak in it: UI-05's output-limit fixture in `lesson-units-api.json` had leaked text in its *request* (the browser's report). The answer was the same, so no test noticed	Re-recording after the fix	Re-recorded; the answer is unchanged
The hint audit could not build a loop with two lines inside it, so lesson 1's approach hint (a `print()` and an input line, both indented) was not tested together	The audit for lesson 1 found no runnable program	`assemble(greedy=True)` keeps every line after a `while`/`for` header indented until the next header; both assemblies are tried. Unit 2's results are unchanged
The test runner had no time limit, so an endless sample hung the suite	Writing the endless-loop samples	`case_runner.py` stops a case after 0.5 s
Near-miss: a test sample meant to loop for ever passed its first case	The sample's verdict was wrong	Rewritten so that it loops on every case

Bug or near-miss	How it was found	Fix
Near-miss: 9 "CSP" console lines in the no-mock run	As in handoffs 24 and 26	My own axe injection, one per scan; the app logs none

6. How it was tested

Commands and results (each suite alone, macOS, Python 3.12.14, Node 24)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db ../../python-academy/backend/.venv/bin/python -m pytest -q`	**633 passed, 11 skipped** (541 before, plus 90 in `test_unit3_lessons.py` and 2 more in `test_recommend.py`)
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**644 passed**
`cd frontend && npx tsc --noEmit && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**205 passed** (173 before, plus 29 in `unit3.spec.ts`, 1 leak test, and 2 more in the course-wide specs)
No-mock run: real API (uvicorn), production build, real Pyodide, each lesson's pass, wrong answer, Python error, endless loop and unsupported import, the ending, and axe-core 4.10.3 (WCAG 2.0/2.1/2.2 A and AA, best practices) on every card and result, the course page and the catalog, at 1280 px and 375 px	Every result as expected; the ending says "released so far"; **0 axe violations**; no sideways scrolling

What the tests cover

Required test	Where
Content validation	Each lesson: browser check v1, 3-second limit, at least 3 visible and 2 hidden cases, no `require`/`forbid`, three hints, a similar example, two other ways, a new badge; fading; expected outputs under 200 characters
What the cards say is true	Every worked example's output; 10 claims on cards and hints, and the predict answers, run in Python (including "deleting the countdown line prints 3 for ever")
Correct programs pass	The reference, creative solutions (`while True` with `break`, a stored flag, `for` with `break`, `%` filtering) and both other ways per lesson
Wrong programs fail with the right result	34 samples: endless (output limit and timeout), too many `input()` calls (missing input), off-by-one limits and ranges, `if` for `while`, hard-coded and visible-only answers; every case rule has a sample

Required test	Where
Hint audit, extended for loops	Code in the hints, tagged hints and similar example, alone and combined, with multi-line loop bodies, with and without the starter; again with names swapped for the challenge's; a control proves the audit catches a hint that spells out the loop
Hidden output never sent	Plans and answers (backend and browser), and ENG-03's leak tests
End to end with real Pyodide, each lesson	Every card; a pass with the badge and other ways; needs_work with the tagged hint first (including lesson 1's "out of replies" with the `EOFError` hint); a Python error with its friendly explanation (`NameError`; `IndentationError` with `indent-loop`; `TypeError`); an endless loop stopped at 3 s; an unsupported import; Try it running out of replies; Try it printing for ever
XP and badges once	300 XP and 3 badges, nothing for a failed attempt, nothing twice
Units 1 and 2 unchanged	`unit1-unchanged.spec.ts` unchanged and passing; `/api/lessons` equals the frozen fixture; all unit 2 tests pass with only the course-wide counts changed
Unit state	Ending "released so far"; `complete: false`; the recorded home note; unit 4 never named; "3 of 12 units open"
axe at desktop and phone	The no-mock run, plus the phone browser test

Mocks: the browser tests answer API calls with recorded **real** answers, and only for an exactly matching request (`test_the_recorded_answers_are_the_real_apis`). Python, the worker and the page are real.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. Lesson 1's approach hint spells out the whole loop	**4**: the hint audit, the renamed audit, and both recordings tests
B2. Lesson 2 without `extra-replies` and `out-of-tries`	**7**: wrong samples now pass (no limit, `<=`), the case-rule samples, content validation and the recordings
B3. The check plan sends hidden expected output	**16**: ENG-03's leak tests, unit 2's and unit 3's leak tests, the plan-structure test and the recordings
B4. The worker's flush removed	**3** Playwright: the new leak test, UI-05's output cap test, unit 3's "prints for ever"
B5. `more_lessons_coming` removed from unit 3	**11**: catalog, ending, recommendation, recordings and content tests

7. Known limitations

- **Unit 3 is not complete**: running totals and nested loops (part 2) are not written. No unit project or unit check.
- **A silent endless loop in Try it takes 15 seconds to stop** (the worker's existing limit). The page stays usable; checks stop at 3 seconds.
- `for` **is taught, not enforced**: a `while` loop passes lesson 3.
- **The new timeout text applies to every unit's checks.**
- **Not tested with a real screen reader**, or on Safari or Firefox. Reading level was written for grade 9, not measured.

8. Live checks for you (after merging and deploying)

You have one test account; your sister can use `/preview` with no account.

- [ ] **The catalog.** `/courses/py`: unit 3 "Repeat and explore" is released with three lessons and the summary "while and for loops that always finish. Running totals and loops inside loops come next."; the catalog says 3 of 12 units open.
- [ ] **Lesson 1, Try it.** In "Keep guessing", card 4: give three wrong replies and run. "Not enough player replies", and the page still works.
- [ ] **Lesson 1, challenge.** Use `if` instead of `while` and run: "Not yet", with a hint about the loop. Fix it: "Check passed" and the Patient guesser badge.
- [ ] **Lesson 2, endless loop.** In "Three tries", card 4: delete `count = count - 1` and run. "Too much output" within a few seconds.
- [ ] **Lesson 2, challenge.** Write a loop with `tries <= 3` (counter starting at 1): "Not yet" for guesses of 1, 2, 3 and 5, and the hint says the loop needs a limit on tries. Fix it: "Check passed".
- [ ] **Lesson 3.** In "Step by step", write `range(2, top, 2)`: "Not yet" for a top of 8. Fix it: "Check passed"; the other ways show a `while` version.
- [ ] **A check that never ends.** In any challenge, write `while True: pass` and run the check: stopped after about 3 seconds, with "Check that something inside the loop changes".
- [ ] **The ending** after lesson 3: "You finished every lesson released so far" (not "Unit complete").
- [ ] **The home page** after that: "Python Foundations unit 3, Repeat and explore, has more lessons on the way. You have finished every lesson in it so far." No unit 4.
- [ ] **XP.** 100 XP and one badge per lesson; passing again adds nothing.
- [ ] **Units 1 and 2 unchanged.**
- [ ] **For your sister:** is the difference between lesson 1 (keep going until right) and lesson 2 (stop after three) clear? Does lesson 3's "up to and including" read clearly?
- [ ] **No CSP reports** in DevTools → Console.

9. Questions for you

1. **Assumption 1, the story** carried on from unit 2. OK? *Recommended, not yet confirmed: keep it for part 2 too (a running score fits).*

2. **Construct rules later.** Should part 2 (nested loops) be the place to add a construct rule, since nesting can be forced by behaviour less easily? *Recommended, not yet confirmed: no rules for part 2 either; revisit with the unit check.*
3. **Assumption 2, the worker fix** is in this PR. Give it its own backlog row? *Recommended, not yet confirmed: yes, as a done ENG-03 follow-up row pointing at this handoff, so it is findable.*
4. **Assumption 3, the timeout wording** now names loops in every unit. OK? *Recommended, not yet confirmed: keep it; endless loops are the usual cause everywhere.*
5. **Next:** unit 3 part 2 (running totals, nested loops)? *Recommended, not yet confirmed: yes, before a unit check, so the unit is whole.*
6. **Still open from handoff 24:** question 6 (one badge per lesson against `course-catalog.md` 6.2).

10. For your docs PR (not edited here)

Where	What
Backlog EDU-03	in_review for part 1 (this PR, handoff 27); part 2 not started
Backlog (optional)	The worker leftover-output fix (question 3)
Stage table	"27. EDU-03 part 1 | Unit 3 lessons 1 to 3, more lessons coming | in_review (handoff 27)"
Decision log	A chapter from sections 4 to 6 of this handoff

11. Next concrete action

Review and merge, run the live checks, then answer question 5.

12. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-u03 status --short   # expect nothing, or only ??
frontend/AGENTS.md
rm -f ../python-academy-u03/frontend/AGENTS.md   # generated by Next.js, not part of the
project
git worktree remove ../python-academy-u03     # deletes only that folder; refuses if
anything is uncommitted
git worktree prune
git branch -d content/py-u03-part1            # refuses if the branch is not merged
```

Don't delete the folder in the Finder or with `rm -rf`; Git would keep a stale record of it (`git worktree prune` fixes that). If `git branch -d` refuses after a squash merge, check the PR says "Merged" on GitHub,

then use `git branch -D content/py-u03-part1`.

Appendix. Every card, hint and check case, word for word

A.1 `py.while-loops`: Keep guessing

Concept: Loops · while. Description: Let the player keep guessing the secret number until they get it right. Badge: **Patient guesser** (`py.badge.patient-guesser`, practice).

Card	Text
1. Discover: Repeat while something is true	A while loop repeats a block of code for as long as its condition is True. Python checks the condition, runs the indented block, then checks again. When the condition becomes False, the loop ends and the program carries on below it. Something inside the loop must change, or the condition stays True forever. To keep asking the player, call input() once before the loop and again at the end of the block. If the condition is False the very first time, the block never runs at all.
2. Worked example	`# Keep asking until the password is moon` / `word = input("Password? ")` / `while word != "moon":` / `print("The door stays shut.")` / `word = input("Password? ")` / `print("The door opens.")` Replies: Reply 1: 'star', Reply 2: 'sun', Reply 3: 'moon'. Output: `The door stays shut.` / `The door stays shut.` / `The door opens.` The condition is checked before each repeat. star and sun are not moon, so the block runs twice. When word becomes moon, the condition is False, the loop ends, and the last line runs.
3. Predict	If the first reply were moon, what would the program print? Options: **Only The door opens.** (answer); The door stays shut. once, then The door opens.; Nothing at all Why: word != "moon" is False straight away, so the block never runs. Python goes on to the last line and prints The door opens.
4. Try it	Run it, then change the replies so that moon comes first, then last. Predict how many times The door stays shut. appears before each run. Then try three wrong replies and read what happens when the replies run out.
5. Challenge	The riddle keeper lets the player keep guessing. The starter asks for the secret number and the first guess. While the guess is wrong, print True if it is too high or False if it is too low, then ask for another guess. When the guess is right, the loop ends. Print just the True and False lines. The check tries several runs of guesses, including some you can't see. Starter: `# These two lines ask for the secret number and the first guess` / `secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `# While the guess is wrong: print whether it is too high, then ask again`

Hint	Text
A nudge	You need a loop that repeats while the guess is not the secret. Inside it, two things happen: a print() and a new guess.
A bigger hint	Look at the comment. The loop's condition compares guess with secret using !=. The last line inside the loop asks for the next guess, the same way the starter asked for the first one.
The approach	A pattern from the worked example: while word != "moon": with an indented print(), then an indented line that asks again. Your loop compares numbers instead of words.
About `EOFError`	Your loop asked for another guess after the last reply. Check that it stops as soon as the guess is right: that is when its condition should become False.
About `NameError`	Use the names from the starter, secret and guess, and spell them the same inside the loop.
About `IndentationError`	The lines inside the loop start with four spaces, so Python knows they belong to it.
About `TypeError`	Each new guess needs int(...) too, like the first one. Python can't compare text with a number using >.
About `py.while-loops.case.three-tries`	With a secret of 7 and guesses of 9, 3 and 7, the lines are True, then False. If only True printed, your program checked once with if; a while loop checks again after every guess.
About `py.while-loops.case.first-try`	When the first guess is right, the loop never runs and nothing is printed. If your program printed a line, check that the print() is inside the loop.
About `py.while-loops.case.many-low`	Three low guesses print three False lines. Make sure your loop keeps going for as long as the guess is wrong, however many guesses that takes.
About `py.while-loops.case.below-zero`	Guesses below zero compare the same way. Compare guess with secret, not with particular numbers.
About `py.while-loops.case.long`	Your loop needs to keep going for as many wrong guesses as the player makes, not for a fixed number of them.
Similar example	`steps = 1` / `while steps < 50:` / `steps = steps * 3` / `print(steps)` The loop keeps tripling steps while it is under 50: 1, 3, 9, 27, then 81. 81 < 50 is False, so the loop stops and 81 is printed. Your challenge repeats while a guess is wrong, and asks for a new guess each time.
Other way, after passing	`secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `while guess != secret:` / `too_high = guess > secret` / `print(too_high)` / `guess = int(input("Your guess? "))` Keep the answer in a variable, then print it.

Hint	Text
Other way, after passing	`secret = int(input("Secret number? "))` / `while True:` / `guess = int(input("Your guess? "))` / `if guess == secret:` / `break` / `print(guess > secret)` while True repeats until break leaves the loop, so input() is written only once. The next lesson looks at how loops end.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`three-tries`	visible	a secret of 7 and guesses of 9, 3 and 7	'7', '9', '3', '7'	`True` / `False`
`first-try`	visible	a secret of 5, guessed first time	'5', '5'	(nothing)
`many-low`	visible	a secret of 10 and guesses of 1, 2, 3 and 10	'10', '1', '2', '3', '10'	`False` / `False` / `False`
`below-zero`	hidden	a secret below zero	'-4', '0', '-9', '-4'	`True` / `False`
`long`	hidden	a long run of guesses	'50', '90', '10', '60', '40', '55', '50'	`True` / `False` / `True` / `False` / `True`

A.2 `py.loop-termination`: Three tries

Concept: Loops · stopping on time. Description: Give the player three tries, and make sure the loop always ends. Badge: **Loop closer** (`py.badge.loop-closer`, practice).

Card	Text
1. Discover: Every loop needs a way out	A loop ends only when its condition becomes False. Something inside the loop has to move it there: a counter that goes up or down, or a new reply. If nothing changes, the loop runs forever. A counter often decides how many times a loop runs. Starting it at 0 or at 1, and choosing < or <=, changes the count by one. To stop for either of two reasons, join the conditions with and: keep going while the guess is wrong and tries are left. Here, a program that never ends is stopped for you after a few seconds.
2. Worked example	`# Count down to the gate opening` / `count = 3` / `while count > 0:` / `print(count)` / `count = count - 1` / `print("open")` Replies: none. Output: `3` / `2` / `1` / `open` count goes 3, 2, 1. After 1 is printed it becomes 0, so count > 0 is False and the loop ends. The last line runs once, after the loop.

Card	Text
3. Predict	If the line count = count - 1 were deleted, what would happen? Options: It would still print 3, 2, 1 and open; **It would print 3 again and again and never finish** (answer); Python would stop with an error at once Why: count would stay 3, so count > 0 would always be True and the loop would never end. Because it prints each time, the academy stops it as soon as the output gets too long. You can try it on the next card.
4. Try it	Predict, then run: change count = 3 to count = 5. Next, delete the line count = count - 1 and run it; the academy stops the program and explains why. Put the line back with Reset code. Last, change > to >= and count how many numbers print.
5. Challenge	The riddle keeper allows three tries. Ask for the secret number, then ask for guesses until the guess is right or three guesses have been made. After the loop, print two lines: how many guesses were made, then True if the last guess was right or False if it was not. Print just the two answers. Starter: `# Ask for the secret number and the first guess` / `# Keep asking while the guess is wrong and tries are left` / `# Then print the number of guesses, and whether the last one was right`

Hint	Text
A nudge	Count the guesses as you go. The loop needs two reasons to keep going, joined with and: the guess is wrong, and fewer than three guesses have been made.
A bigger hint	Check your counter: after the first guess it should be 1. Each time the loop asks again, add 1. With three tries, the loop can ask at most two more times.
The approach	A pattern with other names: while answer != key and turns < 5: with indented lines that ask again and add 1 to turns.
About `EOFError`	Your loop asked for another guess after the last reply. It should stop after three guesses, even when none of them is right: check the counter in the loop's condition.
About `NameError`	Give every name a value before the loop uses it, including the counter, and spell it the same each time.
About `py.loop-termination.case.third-try`	Here the third guess is right, so the lines are 3 and True. If your program stopped after two guesses, check where your counter starts and whether you used < or <=.
About `py.loop-termination.case.out-of-tries`	Three wrong guesses use up the tries, so the second line is False. Print whether the last guess equals the secret, rather than True every time.
About `py.loop-termination.case.first-try`	When the first guess is right, the loop never runs: one guess, and True. Check that your counter is 1 after the first guess.

Hint	Text
About `py.loop-termination.case.extra-replies`	Only the first three guesses count, so the lines are 3 and False, even though a fourth reply would be right. If your program printed 4 and True, the loop needs a limit on tries.
About `py.loop-termination.case.second-try`	Count every guess, including the first one, so the number printed is right whichever try solves the riddle.
About `py.loop-termination.case.below-zero`	A secret below zero works the same way. Compare the guess with the secret instead of checking for particular numbers.
Similar example	`fuel = 10` / `trips = 0` / `while fuel >= 3 and trips < 2:` / `fuel = fuel - 3` / `trips = trips + 1` / `print(trips)` / `print(fuel)` The cart keeps going while there is fuel for a trip and fewer than 2 trips are done. Both reasons are checked each time, and the loop stops when either one fails: here after 2 trips, with 4 fuel left. Your challenge stops when the guess is right or the tries run out.
Other way, after passing	`secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `tries = 1` / `while guess != secret and tries < 3:` / `guess = int(input("Your guess? "))` / `tries += 1` / `print(tries)` / `print(guess == secret)` tries += 1 is a short way to write tries = tries + 1.
Other way, after passing	`secret = int(input("Secret number? "))` / `tries = 0` / `while tries < 3:` / `guess = int(input("Your guess? "))` / `tries = tries + 1` / `if guess == secret:` / `break` / `print(tries)` / `print(guess == secret)` A loop that counts to three, with break to leave early when the guess is right.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`first-try`	visible	a secret of 7, guessed first time	'7', '7'	`1` / `True`
`third-try`	visible	a secret of 7 and guesses of 1, 2 and 7	'7', '1', '2', '7'	`3` / `True`
`extra-replies`	visible	a secret of 5 and guesses of 1, 2, 3 and 5	'5', '1', '2', '3', '5'	`3` / `False`
`out-of-tries`	visible	a secret of 7 and guesses of 1, 2 and 3	'7', '1', '2', '3'	`3` / `False`
`below-zero`	hidden	a secret below zero	'-3', '-1', '-3'	`2` / `True`
`second-try`	hidden	solved on the second try	'20', '25', '20', '99'	`2` / `True`

A.3 `py.for-range`: Step by step

Concept: Loops · for and range(). Description: Count the steps of a staircase with a for loop and range(), two at a time. Badge: **Step counter** (`py.badge.step-counter`, practice).

Card	Text
1. Discover: Count with for and range()	A for loop runs its block once for each value it is given. range(1, 4) gives 1, 2 and 3: it starts at the first number and stops before the second. So for n in range(1, 4): with print(n) inside prints 1, 2 and 3. range(4) on its own gives 0, 1, 2 and 3. A third number sets the step: range(2, 9, 2) gives 2, 4, 6 and 8, and a step of -1 counts down. To include a number at the end, make the stop one bigger. A for loop moves its own variable on each time, so it always ends.
2. Worked example	`# The bridge has 5 planks: number each one` / `for plank in range(1, 6):` / `print(plank)` / `# Then count down from 3 to 1` / `for n in range(3, 0, -1):` / `print(n)` Replies: none. Output: `1` / `2` / `3` / `4` / `5` / `3` / `2` / `1` range(1, 6) stops before 6, so the planks are 1 to 5. A step of -1 counts down: range(3, 0, -1) gives 3, 2 and 1, and stops before 0.
3. Predict	What would for n in range(2, 7, 2): with print(n) inside print? Options: **2, 4 and 6** (answer); 2, 4, 6 and 8; 2 and 4 Why: It starts at 2 and adds 2 each time: 2, 4, 6. The next number would be 8, but range stops before 7, so 8 is never reached.
4. Try it	Predict, then run: change range(1, 6) to range(1, 11). Then make the countdown start at 10. Last, change the countdown's step to -2 and predict the numbers before you run.
5. Challenge	The hero climbs the riddle keeper's staircase two steps at a time, starting on step 2. Ask for the number of the top step, then print every step the hero lands on, up to and including the top step if they land on it. Print just the step numbers, one per line. Starter: `# Ask for the top step, then print every step the hero lands on`

Hint	Text
A nudge	The hero lands on 2, 4, 6 and so on. A for loop over a range() with a step of 2 gives exactly those numbers.
A bigger hint	range() stops before its second number. For a top step of 8, the stop has to be bigger than 8, or step 8 is missed.
The approach	A pattern with other names: for seat in range(1, rows + 1, 3): gives 1, 4, 7 and so on, up to rows. Your loop starts at a different number and steps by 2.
About `TypeError`	range() needs whole numbers. Turn the reply into a number with int(...) before you use it.
About `NameError`	Use the same name for the top step everywhere, and give it a value with input() before the loop.
About `IndentationError`	The line inside the for loop starts with four spaces.
About `py.for-range.case.even-top`	With a top step of 8, the hero lands on 2, 4, 6 and 8. If 8 is missing, the stop of your range() is 8: make it one bigger.

Hint	Text
About `py.for-range.case.odd-top`	With a top step of 7, the hero lands on 2, 4 and 6, never on 8, which is above the top. Check the stop of your range().
About `py.for-range.case.none`	With a top step of 1, the hero never lands on a step, so nothing is printed. A range() that starts above its stop gives no numbers at all.
About `py.for-range.case.bigger`	Use the reply in your range(), not numbers typed into the code, so it works for every staircase.
About `py.for-range.case.odd-bigger`	Check odd top steps as well as even ones: the last step printed is the biggest even number that is not above the top.
Similar example	`for floor in range(10, 0, -3):` / `print(floor)` Starting at floor 10 and going down 3 at a time gives 10, 7, 4 and 1. The stop, 0, is never printed. Your challenge counts up by 2 and has to reach the top step.
Other way, after passing	`top = int(input("Top step? "))` / `step = 2` / `while step <= top:` / `print(step)` / `step = step + 2` The same steps with a while loop. for and range() do the counting for you, so they are the tidy way, but this works too.
Other way, after passing	`top = int(input("Top step? "))` / `for step in range(1, top + 1):` / `if step % 2 == 0:` / `print(step)` Go through every step and print only the even ones: step % 2 == 0 means no remainder after dividing by 2.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`even-top`	visible	a top step of 8	'8'	`2` / `4` / `6` / `8`
`odd-top`	visible	a top step of 7	'7'	`2` / `4` / `6`
`none`	visible	a top step of 1	'1'	(nothing)
`bigger`	hidden	a taller staircase	'20'	`2` / `4` / `6` / `8` / `10` / `12` / `14` / `16` / `18` / `20`
`odd-bigger`	hidden	a tall staircase with an odd top step	'15'	`2` / `4` / `6` / `8` / `10` / `12` / `14`